

Technical Documentation

wasserstein.py — Sliced Wasserstein k-means for financial regime detection

This document describes every class and function in `wasserstein.py`, the module implementing the sliced Wasserstein (SW) k-means clustering pipeline with UnifOrtho orthogonal projections. The focus is on **what each routine computes**, **how it maps onto the mathematics** of the thesis, and — in particular — **how the heavy numerical work is vectorized** with NumPy so the algorithm scales to twenty years of multi-asset data.

Two clustering variants are implemented: the original SW clustering of Luan & Hamp (2025) and the UnifOrtho-projection variant building on Rowland et al. (2019) and Bardenet et al. (2025).

Notation used throughout

Symbol	Meaning	Typical NumPy shape
N	Number of log-return observations	(N, d)
d	Asset dimension (number of series)	—
h1	Lifting window length (atoms per distribution)	—
h2	Lifting stride between windows	—
M	Number of lifted windows / empirical distributions	(M, ...)
L	Number of projection directions (slices)	(d, L)
K	Number of clusters / regimes	(K, ...)
p	Wasserstein order (1 = median, 2 = mean)	—

Contents

1. Module overview & data flow
2. Core distribution classes
 - 2.1 EmpiricalDistribution
 - 2.2 ProjectedDistribution
3. sliced_wasserstein_distance
4. sliced_wasserstein_compute_barycenter
5. lifting_transformation
6. unifortho_projection_vectors / _opt (UnifOrtho generation)
7. sliced_wasserstein_clustering_conv_loop / _opt (k-means)
8. Pipeline drivers: clustering_unifortho, max_acc, max_mccd
9. choose_label (regime canonicalisation)
10. compute_implied_proba (softmax + Bayes + gradient)
11. Vectorization cheat-sheet

1. Module overview & data flow

The module turns a raw price series into regime labels through a fixed sequence of transformations. Understanding this pipeline makes the individual functions easy to place:

- **Prices** → **log-returns**. `r_S = np.diff(np.log(S), axis=0)` gives an (N, d) return matrix.
- **Lifting**. `lifting_transformation` slices the return matrix into M overlapping windows of length $h1$, producing an $(M, h1, d)$ tensor. Each window becomes one d -dimensional empirical distribution μ_m .
- **Projection**. `unifortho_projection_vectors_opt` draws L orthogonal directions and projects every window onto each direction, giving sorted 1-D atom arrays of shape $(M, L, h1)$.
- **Clustering**. `sliced_wasserstein_clustering_conv_loop_opt` runs SW-2 k-means over those projected distributions, returning centroids and a label per window.
- **Post-processing**. `choose_label` assigns bull/bear semantics and `compute_implied_proba` turns distances into soft regime probabilities and a switch signal.

Note Almost every routine exists in a readable reference form and an `_opt` vectorized form that returns the same result. The reference versions document intent; the `_opt` versions are what the pipeline actually calls. The bulk of this document explains how the `_opt` versions remove Python loops.

2. Core distribution classes

2.1 EmpiricalDistribution

EmpiricalDistribution(`l_m: np.ndarray`)

Wraps a single d -dimensional empirical measure supported on $h1$ atoms. It stores the support `l_m` of shape $(h1, d)$ and uniform weights $1/h1$. Its one piece of behaviour is `project(theta)`, the projected empirical measure of Eq. (7) in the thesis.

```
def project(self, theta):
    return np.sort(self.l_m @ theta)
```

The matrix-vector product `l_m @ theta` computes the inner products $\langle x_i, \theta \rangle$ for all $h1$ atoms at once (one BLAS call instead of an $h1$ -length loop). Sorting immediately is the key trick: in one dimension the Wasserstein distance between two equally-weighted empirical measures is just the L^p distance between their *sorted* atoms, so sorting here means every later distance is a plain array subtraction.

2.2 ProjectedDistribution

ProjectedDistribution(`x_points: np.ndarray`)

Represents a 1-D projected measure (the atoms after projection onto one direction). On construction it caches the sorted atoms, mean and variance so repeated clustering iterations never re-sort. Accessors `return_sorted_atoms()`, `return_mean()` and `return_variance()` expose the cached values. This caching is what lets the clustering loop later stack everything into one big array cheaply.

3. sliced_wasserstein_distance

`sliced_wasserstein_distance(proj_dist_1, proj_dist_2, p).compute_distance_matrix()`

Computes the SW-p distance between two sets of projected distributions (one per direction). Each input is a length-L list of ProjectedDistribution objects; the two lists must share the same L and h1.

Vectorization

The reference idea would loop over the L projections, compute a 1-D Wasserstein distance for each, and average. Instead the method stacks the pre-sorted atoms into two dense matrices and works on all projections simultaneously:

```
X1 = np.array([d.return_sorted_atoms() for d in self.dist1]) # (L, h1)
X2 = np.array([d.return_sorted_atoms() for d in self.dist2]) # (L, h1)

if self.p == 1:
    return float(np.mean(np.mean(np.abs(X1 - X2), axis=1)))
elif self.p == 2:
    return float(np.mean(np.sqrt(np.mean((X1 - X2)**2, axis=1))))
else:
    return float(np.mean(np.mean(np.abs(X1 - X2)**self.p, axis=1)**(1/self.p)))
```

$X1 - X2$ is a single (L, h1) elementwise subtraction. `axis=1` collapses the h1 atoms, giving the per-projection 1-D distance; the outer `np.mean` averages over the L slices, matching the Monte-Carlo SW approximation of Eq. (9). For $p = 2$ the inner step is the closed-form 1-D W_2 (root-mean-square gap of sorted atoms):

$$W_2(\mu'_i, \nu'_i) = \sqrt{(1/h1) \sum_i (X1_{1,i} - X2_{1,i})^2}, \quad SW \approx (1/L) \sum_i W_2$$

Note Convention worth checking against your thesis. The code returns $mean_i W_2$ (average of per-slice distances). The most common textbook definition of SW_2 is $(mean_i W_2^2)^{1/2}$ — raise to the power p inside the average, then take the p-th root once at the end. Both are valid metrics and your Eq. (8) actually matches the code, but Eq. (10) raises W to the power p. Pick one convention and use it consistently in the distance, the barycenter objective and the MCCD metric, or the numbers will not line up across chapters.

4. sliced_wasserstein_compute_barycenter

`sliced_wasserstein_compute_barycenter(projected_distributions, p) -> list`

Computes the SW-p barycenter of a group of M_k projected distributions (the cluster members). Input is an M_k -length list, each element itself a length-L list of ProjectedDistribution. Output is a single length-L list representing the centroid, one 1-D distribution per direction.

Vectorization

```
X = np.array([[d.return_sorted_atoms() for d in dist_list]
              for dist_list in projected_distributions]) # (M_k, L, h1)

if p == 1:
    centroid_points = np.median(X, axis=0) # (L, h1)
else:
    # p == 2
    centroid_points = np.mean(X, axis=0) # (L, h1)

return [ProjectedDistribution(centroid_points[l]) for l in range(L)]
```

All members are stacked into one (M_k , L, h1) tensor and reduced over `axis=0` in a single call. This uses the standard fact that in 1-D the W_2 barycenter of equal-weight measures is the **pointwise mean of the sorted atoms** (the quantile average), and the W_1 barycenter is the pointwise **median**. Because the atoms were already sorted at construction, no re-sorting is needed: averaging the i-th order statistic across members directly yields the i-th order statistic of the barycenter.

5. lifting_transformation

`lifting_transformation(r_S, h1, h2) -> np.ndarray`

Implements the sliding-window lifting of Eq. (3): it turns an (N, d) return series into an $(M, h1, d)$ stack of overlapping windows, where window m holds $h1$ consecutive observations and successive windows are $h2$ apart.

```
windows = sliding_window_view(r_S, window_shape=h1, axis=0) # (N-h1+1, d, h1)
windows = windows.transpose(0, 2, 1)                        # (N-h1+1, h1, d)
lifted_samples = windows[:, :h2]                            # stride by h2
return lifted_samples.copy()
```

Vectorization

- `sliding_window_view` builds *all* overlapping windows as a **zero-copy view** using stride tricks — no data is duplicated and no Python loop runs. NumPy returns the window axis last, hence shape $(N-h1+1, d, h1)$.
- `transpose(0, 2, 1)` reorders to the (window, time, asset) layout the rest of the code expects.
- `[:, :h2]` applies the stride, keeping every $h2$ -th window.
- `.copy()` materialises a contiguous array. This matters: the view aliases the original buffer, and the later tensor projection / sort run far faster on contiguous memory (and avoids subtle bugs if `r_S` is mutated).

Note Off-by-one to verify. The view yields $N-h1+1$ raw windows, then the stride keeps $\lfloor (N-h1+1)/h2 \rfloor$ of them. Elsewhere the code sets $M = \text{floor}((N - (h1 - h2)) / h2)$. Confirm these two counts agree for your $h1, h2$ (they coincide in the common case but can differ by one at the boundary), since M is used to size loops over the lifted windows downstream.

6. UnifOrtho projection generation

`unifortho_projection_vectors(S, K, L, h1, h2) / ..._opt(...)`

Builds the projected empirical distributions for every window. Both versions (a) take log-returns, (b) lift them, (c) generate L orthogonal directions via repeated QR of Gaussian matrices, and (d) project + sort. They differ only in step (d). The result has logical shape (M, L) : for each window, L sorted 1-D projections.

Generating Haar-orthogonal directions

```
d = r_S.shape[1]
k = math.ceil(L / d)                # number of d x d orthogonal bases needed
theta = []
for i in range(k):
    Z = np.random.normal(size=(d, d))
    Q, R = np.linalg.qr(Z)           # O(d^3)
    lambda_i = np.diag(np.sign(np.diag(R))) # sign-fix diagonal
    U_i = Q @ lambda_i               # Haar-distributed on O(d)
    theta.extend(U_i.T)
theta = np.array(theta).T[:, :L]    # (d, L): keep exactly L directions
```

This is the Mezzadri (2007) recipe used in your Algorithm 1. A raw QR of a Gaussian matrix is not quite Haar-uniform on $O(d)$; multiplying Q by the sign of R 's diagonal corrects the phase so the columns of U_i are genuinely uniform orthonormal directions. Each basis supplies d mutually orthogonal slices, so only $\lceil L/d \rceil$ QR calls are needed to obtain L directions — this orthogonality is the variance reduction the thesis relies on versus i.i.d. Monte Carlo.

The vectorized projection (the heart of the `_opt` version)

The reference version loops over windows and directions, calling `emp_dist[m].project(theta[:, l])` for every (m, l) pair — $M \times L$ Python calls, each doing its own sort. The `_opt` version replaces the whole double loop with three array ops:

```

l_r_S = lifting_transformation(r_S, h1, h2) # (M, h1, d)
theta = ...                               # (d, L)

# 1. one batched matmul projects every window onto every direction
projections = l_r_S @ theta                # (M, h1, d)@(d, L) -> (M, h1, L)

# 2. sort the atoms (axis=1 = the h1 time atoms)
sorted_projections = np.sort(projections, axis=1) # (M, h1, L)

# 3. reorder to (M, L, h1) to match the (window, direction, atom) layout
sorted_projections = sorted_projections.transpose(0, 2, 1)

projected_emp_dist = [[ProjectedDistribution(sorted_projections[m, l])
                        for l in range(L)] for m in range(M)]

```

- **Batched matmul.** $(M, h1, d) @ (d, L)$ broadcasts the matrix product over the leading M axis, so all $M \times h1 \times L$ inner products are computed in a single BLAS-backed call instead of $M \times L$ separate projections.
- **Vectorized sort.** `np.sort(..., axis=1)` sorts every (window, direction) column of atoms at once. Sorting here, once, means the clustering loop never sorts again.
- **Layout transpose.** The final `transpose(0, 2, 1)` matches the downstream $(M, L, h1)$ convention so object construction is a clean double comprehension.

Note The remaining list-comprehension that wraps slices in `ProjectedDistribution` objects is the one part still in Python. If profiling shows it dominating for large $M \cdot L$, you can keep `sorted_projections` as a raw array and skip the objects entirely — the `_opt` clustering loop already re-extracts a raw array anyway (see §7).

7. SW k-means clustering loop

`sliced_wasserstein_clustering_conv_loop(...)` / `..._opt(...)`

The Lloyd-style k-means over projected distributions. Both versions initialise K centroids from random windows, then alternate an **expectation** step (assign each window to its nearest centroid under SW-2) and a **maximisation** step (recompute each centroid as the SW-2 barycenter of its members), stopping when total centroid movement falls below `epsilon` or after 50 iterations.

Reference version

The readable version expresses the algorithm directly with object-level calls: a Python loop over the M windows, each computing K `sliced_wasserstein_distance(...).compute_distance_matrix()` values, then `np.argmin`; the maximisation gathers members per cluster and calls the barycenter helper. Clear, but the assignment step is $O(M \cdot K)$ Python iterations per sweep.

Optimized version — pre-extraction

Before the loop, every projected distribution is pulled once into a dense tensor so the iteration touches only NumPy arrays:

```
h1 = len(projected_emp_dist[0][0].return_sorted_atoms())
X = np.empty((M, L, h1))
for m in range(M):
    for l in range(L):
        X[m, l, :] = projected_emp_dist[m][l].return_sorted_atoms()
```

Optimized version — vectorized expectation (broadcasting)

Centroids are extracted into C of shape $(K, L, h1)$, then the full (M, K) distance matrix is produced with one broadcast subtraction — no loop over windows or clusters:

```
# X[:, None, :, :] -> (M, 1, L, h1)  C[None, :, :, :] -> (1, K, L, h1)
diff_sq = (X[:, None, :, :] - C[None, :, :, :]) ** 2      # (M, K, L, h1)

# SW-2: rms over atoms (axis 3), then mean over directions (axis 2)
sw_dist = np.mean(np.sqrt(np.mean(diff_sq, axis=3)), axis=2)  # (M, K)

labels = np.argmin(sw_dist, axis=1)
```

The two `None` (newaxis) insertions make X and C broadcast against each other into a 4-D $(M, K, L, h1)$ difference: every window is compared with every centroid in one expression. Reducing `axis=3` then `axis=2` reproduces exactly the SW-2 of §3 (rms over atoms, mean over slices), and `argmin(axis=1)` assigns all M labels at once. This single statement replaces the entire $M \times K$ object-distance double loop.

Note Memory cost. The intermediate `diff_sq` is $M \cdot K \cdot L \cdot h1$ floats. For large products this 4-D array can be the binary's peak memory. If it ever gets tight, chunk over M (process windows in batches) or fold the square-and-mean with `np.einsum` to avoid materialising the full tensor.

Optimized version — vectorized maximisation (boolean masks)

```
for k in range(K):
    mask = (labels == k)
    if np.any(mask):
        barycenter_matrix = np.mean(X[mask], axis=0)  # (L, h1)
        centroids[k] = [ProjectedDistribution(barycenter_matrix[l])
                        for l in range(L)]
    else:
        centroids[k] = projected_emp_dist[random.randint(0, M-1)]
```

`X[mask]` is boolean fancy-indexing: it selects the $(M_k, L, h1)$ slab of windows assigned to cluster k , and `np.mean(..., axis=0)` is the 1-D W_2 barycenter (pointwise mean of sorted atoms, as in §4) in a single reduction. Empty clusters fall back to a random re-seed — the standard Lloyd safeguard against dead

centroids.

Optimized version — vectorized convergence test

```
centroid_changes = np.sum(np.mean(np.sqrt(np.mean((C_old - C)**2, axis=2)), axis=1))
if centroid_changes < epsilon:
    break
```

The same SW-2 reduction is applied between old and new centroid tensors and summed over the K clusters, giving total centroid movement in one vectorized expression rather than a per-cluster loop of object distances.

8. Pipeline drivers

sliced_wasserstein_clustering_unifortho(S, K, L, epsilon, h1, h2)

The end-to-end single-run pipeline: compute returns, derive M, build the projected distributions with `unifortho_projection_vectors_opt`, then run the `_opt` k-means. Returns (`projected_emp_dist`, `centroids`, `labels`). This is the function to call for one clustering of a fixed dataset.

max_acc_unifortho_sim(N_S, S, r_s_true_regime, K, L, epsilon, h1, h2, test)

Runs `N_S` random restarts and keeps the run with the highest **accuracy** against ground-truth regimes (via `mt.total_accuracy`). Crucially, it computes the projected distributions **once** and reuses them across all restarts — only the cheap clustering loop is repeated, since the expensive projection/sort does not depend on the random centroid seeding. The optional `test` flag prints per-regime mean / variance / skew / 5% CVaR diagnostics. Intended for **synthetic** experiments where labels are known.

Note Selection bias to acknowledge in the thesis. Choosing the restart by accuracy uses the ground-truth labels, so reported accuracy is an oracle-selected best case, not what you would get unsupervised. That is fine for the synthetic-data sanity checks of Chapter 4, but the real-data strategies correctly use `max_mccd` instead (an unsupervised criterion). Make that distinction explicit so a reader does not read Chapter 4 numbers as out-of-sample.

max_mccd_unifortho_sim(N_S, S, K, L, epsilon, h1, h2, metric)

The unsupervised analogue used by the trading strategies. It keeps the restart with the largest **mean centroid-to-centroid distance** (`mt.mean_centroid_centroid_distance`) — i.e. the most clearly separated clustering — then calls `choose_label` to fix bull/bear semantics. No ground truth is used, so this is legitimate for live/backtest signals.

9. choose_label

choose_label(best_centroids, best_labels, metric, K) -> (centroids, labels)

k-means labels are arbitrary integers; this routine canonicalises them so label 0 is always the bearish regime and label 1 the bullish one, making downstream signal logic deterministic. For each cluster it averages, across the `L` centroid slices, the mean / variance / skew / 5% CVaR of the sorted atoms, then orders the two clusters:

- **metric = "CVaR"**: label 0 is the cluster with the lower (more negative) 5% conditional value-at-risk — the regime with the worse tail.
- **metric = "MeanVar"**: label 0 is the cluster with higher variance and lower mean.

When a swap is needed it exchanges the two centroid lists and flips labels with the vectorized trick `best_labels = 1 - best_labels`. Only `K = 2` is implemented; other `K` print a not-implemented notice.

Note Two things to tidy for the dissertation. (1) The CVaR estimate `sorted_atoms[:int(len*0.05)]` is the 5% left tail of each *centroid's* atoms — document that you take CVaR of the barycenter, not of pooled members. (2) The MeanVar branch leaves labels unchanged when the mean/variance ordering is ambiguous, which can let an un-canonicalised labelling through; consider a deterministic tie-break (e.g. fall back to mean alone).

10. compute_implied_proba

`compute_implied_proba(projected_emp_dist, centroids, labels, tau=None, tau_gradient=None, lookback=5, use_gradient=False, gradient_weight=0.3)`

Turns the geometry of the clustering into soft probabilities and a regime-switch signal for the most recent window. This implements Section 3.3 of the thesis: a softmax likelihood, a Markov prior, a Bayesian posterior and an optional gradient (momentum) term.

Step 1 — distance matrix (same broadcast as k-means)

```
diff_sq = (X[:, None, :, :] - C[None, :, :, :]) ** 2      # (M, K, L, h1)
dist_matrix = np.mean(np.sqrt(np.mean(diff_sq, axis=3)), axis=2) # (M, K)
```

Identical vectorization to §7: every window's SW-2 distance to every centroid in one shot.

Step 2–3 — temperature & softmax likelihood

If `tau` is not supplied it defaults to half the mean inter-centroid distance, adapting the softmax sharpness to the geometry. The likelihood of Section 3.3 is then a numerically-stabilised softmax over negative distances:

```
neg_scaled = -dist_matrix / tau
neg_scaled -= neg_scaled.max(axis=1, keepdims=True) # stability: subtract row max
exp_vals = np.exp(neg_scaled)
proba_matrix = exp_vals / exp_vals.sum(axis=1, keepdims=True) # (M, K)
```

Subtracting the per-row maximum before `exp` prevents overflow without changing the result (softmax is shift-invariant). The whole (M, K) probability matrix is produced vectorized over rows via `keepdims=True` broadcasting.

Step 4 — empirical transition matrix

```
transition_matrix = np.zeros((K, K))
for m in range(M - 1):
    transition_matrix[labels[m], labels[m + 1]] += 1
row_sums = transition_matrix.sum(axis=1, keepdims=True)
row_sums[row_sums == 0] = 1
transition_matrix /= row_sums
```

Counts label transitions into a K×K matrix and row-normalises into the first-order Markov prior $\pi_{i,j}$ of Section 3.3. The guard `row_sums[row_sums == 0] = 1` avoids divide-by-zero for an unvisited regime.

Note Optional vectorization. The transition count loop can be replaced by `np.add.at(transition_matrix, (labels[:-1], labels[1:]), 1)`, which does the same accumulation with no Python loop. Minor here (it is O(M), not the bottleneck) but tidy.

Step 5 — Bayesian posterior & switch probability

```
current_regime = labels[-1]
prior = transition_matrix[current_regime]      # pi[z_current, :]
likelihood = proba_matrix[-1]                # softmax for last window
posterior = prior * likelihood
posterior /= (posterior.sum() + 1e-12)
...
switch_proba = 1.0 - posterior[current_regime]
```

This is Bayes' rule exactly as in the thesis: $\text{posterior} \propto \text{prior} \times \text{likelihood}$, then $P_{\text{switch}} = 1 - P_{\text{Bayes}}(z_{\text{current}})$. The `+ 1e-12` keeps the normalisation safe.

Optional gradient (momentum) term

When `use_gradient=True` and enough history exists, the routine fits an OLS slope β_k of the distance-to-centroid-k over the last `lookback` windows using `np.polyfit(..., deg=1)`. A negative slope means the market is drifting *toward* regime k, so the sign is flipped before a softmax (temperature

`tau_gradient`) turns the slopes into a probability vector. The final posterior blends the two:

$$\text{posterior} = (1 - \alpha) \cdot P_{\text{Bayes}} + \alpha \cdot P_{\text{grad}}, \quad \alpha = \text{gradient_weight}$$

matching the gradient blend of Section 3.3. The function returns `(proba_matrix, switch_proba, transition_matrix, posterior)`.

11. Vectorization cheat-sheet

The recurring NumPy idioms that make the module fast, in one place:

Technique	Where	What it replaces
Sort-on-construction (1-D Wasserstein = sorted-atom Lp gap)	ProjectedDistribution, project()	Re-sorting inside every distance call
Batched matmul (M,h1,d)@(d,L)	unifortho_..._opt	M x L separate projection calls
np.sort(..., axis=1) on the tensor	unifortho_..._opt	Per-window, per-direction sorts
sliding_window_view (zero-copy strides)	lifting_transformation	Python loop building each window
4-D broadcast X[:,None] - C[None,:]	conv_loop_opt, compute_implied_proba	M x K nested distance loop
Reduce axis 3 then axis 2 (rms, mean)	SW-2 everywhere	Manual per-slice averaging
Boolean mask X[labels==k].mean(0)	conv_loop_opt maximisation	Gathering members in a Python list
Shift-invariant stable softmax (subtract row max)	compute_implied_proba	Naive exp (overflow risk)
1 - labels to flip a 2-class labelling	choose_label	Element-wise relabel loop

Suggested next steps

- Settle a single SW-p convention (§3 note) and propagate it through distance, barycenter and MCCD so figures are mutually consistent.
- Reconcile the M count between `lifting_transformation` and the `floor((N - (h1 - h2))/h2)` formula (§5 note).
- Make the Chapter 4 (accuracy-selected) vs strategy (MCCD-selected) distinction explicit (§8 note) to avoid an apparent look-ahead in the synthetic results.

Generated as a companion reference for the dissertation “Regime Detection using Sliced Wasserstein k-means clustering with UnifOrtho”. Code behaviour described as of the supplied `wasserstein.py`.